

WorkForcePro

Bonus Investigation

Cross-Employee OJT Record Visibility — Root Cause Investigation

Document Type: Investigation Report — Part 7 (Bonus/Optional) Deliverable

Project: WorkForcePro QA Take-Home Assignment

Version: 1.0

Prepared by: Puskar Adhikari

Role: Sr. QA Engineer

Submission Date: 2026-07-05

1. Reported Issue

“An employee can see OJT records belonging to another employee.”

This is a user-reported access-control issue: an employee is able to view employee_trainings (OJT) records that do not belong to them. Taken at face value, this is a potential Broken Object-Level Authorization (BOLA/IDOR) issue and is treated as high-severity/high-priority from the moment it is reported, since it implies employee data is visible across account boundaries — a direct violation of the principle that OJT records are personal, supervisor-gated data (OJT-003).

1.1 Why This Matters Beyond a Normal Defect

Unlike a functional defect (e.g., a button not working), a cross-account data visibility issue is a security/compliance concern. It is investigated with three goals in parallel: confirm scope and severity quickly, avoid destructive or exposing actions during investigation, and produce a fix that closes the root cause rather than only the reported symptom.

2. Investigation Approach

The investigation proceeds in four stages, each using a different tool to independently confirm the same fact from a different angle — the point of using Rails Console, SQL, and application logs together is that each is checked against the others, since a single tool's output can mislead. For example, the console might show correctly scoped ActiveRecord code while the actual controller in production runs a different, unscoped code path — the logs are what would reveal that discrepancy.

Stage	Tool	What It Confirms
1. Reproduce & Scope	Manual testing / bug report detail	Exact steps, URLs/IDs involved, whether it's UI navigation or direct ID manipulation
2. Data-Layer Check	Rails Console	What the ORM layer returns for a given user/record combination, and whether model-level scoping exists at all
3. Data Integrity Check	SQL	Whether the underlying data itself is correct (no corrupted employee_id/training_id), independent of any application code
4. Real-World Confirmation	Application Logs	What actually happened in the real request that the user experienced — the authenticated user, the requested ID, and the response returned

3. Step-by-Step Investigation

3.1 Reproduce & Scope the Issue

1. Get the exact repro from the reporting user: which employee account was logged in, what action was taken (e.g., opened a direct link, clicked a UI element, or noticed a record appear on a dashboard), and the specific OJT record(s) seen that shouldn't have been visible.
2. Identify the specific URL, page, or API endpoint involved (e.g., a training detail view at `/trainings/:id` or an API call to `GET /api/trainings/:id`).
3. Confirm whether this is read-only visibility or whether the employee could also take action (e.g., mark someone else's training complete) — the latter is significantly more severe and would be escalated immediately.
4. Confirm the environment (production vs staging) and whether this is reproducible on demand or was a one-time observation, which affects whether this is a consistent code defect or an intermittent issue (e.g., caching).

3.2 Rails Console Investigation

Using a read-only session (rails console --sandbox, or connecting to a read replica if available) to avoid any risk of side effects while investigating a live data issue:

```
# 1. Load the specific record the employee reported seeing
et = EmployeeTraining.find(123)
et.employee_id
# => confirm this does NOT match the reporting employee's own id

# 2. Confirm the reporting employee's own scoped records
reporting_employee = Employee.find_by(employee_id: "EMP-045")
reporting_employee.employee_trainings.pluck(:id)
# => does 123 appear here? It should not, per the report.

# 3. Check whether the model/controller exposes a proper scoping method
reporting_employee.employee_trainings.where(id: 123)
# => if this returns empty but the record was still shown in the app,
#     the controller is NOT using this scoped association – it is likely
#     calling EmployeeTraining.find(params[:id]) directly, unscoped.

# 4. If an authorization gem (e.g., Pundit) is in use, check the policy directly
Pundit.policy(reporting_employee.user, et)&.show?
# => if this returns false but the page still rendered, the controller
#     action is not actually invoking `authorize` at all.

# 5. Check supervisor-based visibility rules, in case the reporting
#     employee is unintentionally linked as a supervisor
reporting_employee.supervisor?
reporting_employee.subordinates.pluck(:id) if reporting_employee.respond_to?(:subordinates)
```

3.3 SQL Query Investigation

Run directly against the database (or a read replica) to confirm the raw data is correct, independent of any application/ORM code — this rules out "is the data itself wrong" versus "is the code showing correct data incorrectly":

```
-- Confirm the record's true owner
SELECT id, employee_id, training_id, status
FROM employee_trainings
WHERE id = 123;

-- Confirm the reporting employee's actual internal id
```

```

SELECT id, employee_id, name
FROM employees
WHERE employee_id = 'EMP-045';

-- Rule out a bad join/reporting view unioning employees together
-- (if any dashboard/report view exists, inspect its definition)
SHOW CREATE VIEW training_dashboard_view; -- or equivalent, if such a view exists

-- Rule out an accidental supervisor/role link causing legitimate-looking
-- but unintended visibility
SELECT *
FROM role_assignments
WHERE employee_id = (SELECT id FROM employees WHERE employee_id = 'EMP-045');

```

3.4 Application Log Investigation

Search centralized logs (Rails production log, or an aggregator such as CloudWatch/Datadog/ELK) for the exact request the reporting employee made, using the approximate timestamp and account they provided:

```

# Example log-search targets (exact command depends on the logging stack):
# - Request path: GET /trainings/123 or /api/trainings/123
# - Authenticated user/session id at time of request
# - HTTP response status code returned (expect 200 – should have been 403/404)
# - Any authorization-related log line (e.g., Pundit::NotAuthorizedError)
#   that should have been raised but was absent

```

- A 200 OK response for a record the reporting employee does not own, with no authorization error logged, strongly indicates the controller action never attempted an authorization check for this endpoint.
- Cross-reference the request timestamp against recent deploys (git log / deploy history) to check whether this is a regression introduced by a specific recent change to this controller action.

4. Possible Root Causes

Ranked by likelihood given the symptom description, most likely first. RC-1 and RC-2 are two sides of the same underlying problem (missing or unenforced object-level authorization) and are the primary suspects; RC-3 through RC-5 are checked to rule out alternate explanations before a fix is finalized.

ID	Root Cause	Likelihood	Rationale
RC-1	Broken Object-Level Authorization (IDOR) — controller loads the record directly by ID (<code>EmployeeTraining.find(params[:id])</code>) without scoping to the current employee's own association.	High	Most common root cause for exactly this symptom in Rails apps; directly explains why changing an ID in a URL/API call surfaces another employee's record.
RC-2	Authorization policy exists but is not invoked in this specific controller action (e.g., a Pundit <code>`authorize`</code> call was omitted when this endpoint was added or refactored).	High	Common when a new action is added without the standard authorization boilerplate, or when <code>`verify_authorized`</code> is not enforced globally.
RC-3	Overly broad supervisor/role check — a 'has supervisor role' check that isn't scoped to 'is THIS employee's supervisor', incorrectly granting broad visibility.	Medium	Plausible if the reporting employee's account has any supervisor-adjacent role; explains a leak that looks like it 'should' be restricted to a hierarchy but isn't scoped correctly.
RC-4	Shared/misconfigured cache key that does not include the requesting user's id, serving a cached response meant for a different employee.	Low	Would typically produce intermittent, not consistently reproducible, symptoms — worth ruling out but less likely to be the sole cause if the issue reproduces reliably.
RC-5	Client-side session/state leakage (e.g., stale front-end state on a shared or improperly logged-out browser session).	Low	Unlikely if reproduced via direct ID/URL manipulation against a properly authenticated session, but worth ruling out via log user-agent/session-id cross-checks.

5. Validation Approach

5.1 Confirming the Root Cause

1. Reproduce definitively with two known test accounts (Employee A, Employee B): log in as A, attempt to view B's known `employee_trainings` record ID via direct URL/API manipulation, and confirm the record is returned.
2. Repeat across at least 2–3 different employee ID pairs to confirm the issue is systemic (a missing scope check) rather than specific to one corrupted record or one misconfigured account.
3. Check adjacent endpoints for the same pattern — certifications and employee profile records use the same `employee_id`-scoped structure, so if this controller action was copy-pasted, the same unscoped `.find(params[:id])` pattern may exist there too.
4. Once RC-1/RC-2 is confirmed via Rails Console (Section 3.2), correlate with the exact log entry (Section 3.4) to confirm the production request matches the console findings — this closes the loop between 'what the code does' and 'what actually happened to the user'.

5.2 Confirming the Fix

1. Add an automated regression test (RSpec request spec) that logs in as Employee A and asserts a 403/404 response when requesting Employee B's `employee_trainings` record — this test must fail against the current code and pass only after the fix.
2. Re-run the manual two-account reproduction from Section 5.1 after the fix is deployed to staging; confirm the same request now returns 403/404 instead of 200.
3. Confirm legitimate access still works: Employee A can still view their own records, and a supervisor can still view their actual subordinates' records — a fix that is too aggressive and blocks legitimate access is also a defect.
4. Monitor authorization-error rates in logs for a short period after production deployment to confirm no unexpected spike (which could indicate the fix is too restrictive) or continued unauthorized 200 responses (which would indicate the fix is incomplete).

6. Fix Recommendations

6.1 Immediate Code Fix

Scope every record lookup to the current employee's own association instead of a bare primary-key lookup:

```
# Before (vulnerable – loads any record by ID regardless of owner):
@employee_training = EmployeeTraining.find(params[:id])

# After (scoped – raises ActiveRecord::RecordNotFound for records outside
# the current employee's own scope, which Rails maps to a 404, avoiding
# even confirming that a given ID exists for another employee):
@employee_training = current_employee.employee_trainings.find(params[:id])

# For supervisors who legitimately need to view a subordinate's record,
# scope to the subordinate relationship explicitly rather than opening
# visibility to all employee_trainings:
@employee_training = current_employee.subordinate_employee_trainings.find(params[:id])
```

6.2 Authorization Layer

- If using Pundit/CanCanCan, ensure `authorize @employee\_training` is called in every controller action that touches this resource, not only the ones added most recently.
- Add `after\_action :verify\_authorized` (Pundit) at the controller level so a missing `authorize` call fails the build/test suite rather than silently shipping — this directly prevents RC-2 from recurring.
- Write the supervisor-visibility policy explicitly as "subordinate.employee_trainings", not "any employee_trainings if current_user.supervisor?" — closing RC-3.

6.3 Process / Prevention

- Add an authorization-test requirement to the PR checklist for any new or modified controller action that loads a record by ID.
- Extend the Master Test Plan's RTM with a new security-focused requirement (e.g., SEC-001: 'Employees can only view their own OJT records; supervisors can only view their subordinates' records'), with a linked automated regression test — the same pattern already used for OJT-001's authorization check (TC-019/TC-020).
- Periodically audit all `\*.find(params[:id])` call sites in the codebase for the same unscoped-lookup pattern, since one confirmed instance often indicates the pattern was copy-pasted elsewhere.

7. Summary

Aspect	Conclusion
Most likely root cause	Broken object-level authorization — record loaded by ID without scoping to the current employee (RC-1), likely combined with a missing/unenforced authorization policy call (RC-2).
Severity	High — cross-account data visibility, potential compliance impact if OJT/training data is considered sensitive personnel information.
Confirmed via	Rails Console (scoped vs unscoped query comparison), SQL (data itself is correct), and Application Logs (real request returned 200 with no authorization error logged).
Fix	Scope all record lookups to the current employee's own association (or subordinate relationship for supervisors); enforce authorization policy checks on every action via <code>`verify_authorized`</code> .
Prevention	PR checklist requirement + automated regression test + new RTM security requirement (SEC-001), following the same pattern already used for OJT-001 authorization testing.